- Because we control the wrapper's interface, it won't change without our knowledge.
- The unit specs for Invoice won't break if the TaxJar API changes.

We do need to have *some* way to detect changes in TaxJar, though. The best place for that is an integration spec for our SalesTax wrapper, which is easy to write because we've kept our wrapper thin:

15-using-test-doubles-effectively/24/sales_tax/spec/integration/sales_tax_spec.rb

```
require 'sales_tax'

RSpec.describe SalesTax do
  let(:sales_tax) { SalesTax.new }

  it 'can fetch the tax rate for a given zip' do
    rate = sales_tax.rate_for('90210')
    expect(rate).to be_a(Float).and be_between(0.01, 0.5)
  end

  it 'raises an error if the tax rate cannot be found' do
    expect {
      sales_tax.rate_for('00000')
    }.to raise_error(SalesTax::RateUnavailableError)
  end
end
```

Because this spec tests against the real TaxJar classes, it will correctly fail if there are breaking changes in the API. Although the API should be stable, we expect the tax rate to fluctuate somewhat. That's why we're checking against a range rather than an exact value.

As a next step, we could use the VCR gem here to cache responses. We could then run these specs without needing real API credentials—which will be handy on our continuous integration (CI) server. We'd still want to revalidate against the real service periodically, which we can easily do by deleting the VCR recordings.

Your Turn

Whew! We gave you a lot to chew on in this chapter, from broad advice on test doubles to fine-grained design nuances. Here are just some of the principles we explored over several code examples:

- Construct your test environment carefully.
- Watch out for the "stubject."
- Know the risks of partial doubles.
- Favor explicit dependency injection over more implicit techniques.
- Avoid faking an interface you don't control.